

Scenarios of Shop Floor Demonstrators

For the circular factory showcase

Key actors

Production Control (PC/A01): The production control is responsible for making decisions on the production system level. The decisions are about specific operations which are then executed by the specific stations, e.g. Remove the lid of Angel Grinder 123 at Station 89 or Transport Bevel Gear 56 by the intralogistic system.

Intralogistic System (ILS/C03): The intralogistics system is responsible for moving goods between stations. It consists of manipulator and transport modules that are moved by mobile robots. Using the manipulator module, assemblies are picked up and transported using the transport modules. Modules cannot move by themselves but are moved by the mobile robots.

Manipulation System (MS/C06): The manipulation system is responsible for picking up the assembly from a station. It is part of the intralogistics system and also part of stations that are in possession of their own robotic arm to pick and place assemblies. The manipulation system executes all manipulation operations.

Inspection System (IS/B01): The inspection system is responsible for performing all measurement tasks in the circular factory. It consists of multiple measurement devices. Besides performing measurements it is able to suggest suitable measurement processes for a given task.

Transformer Cell (TC/C01): The transformer cell is responsible for performing all processes on the products. These processes may include unscrewing, milling and removing parts. It may use robots, process specific modules or refer to speciality machines to execute the process. Processes may fail, which gets detected by the TC. The TC also has an automatic door, which can be opened and closed to provide access to the ILS

Plan Healer (PH/A02): The plan healer PH is responsible for finding new process alternatives, if and when the current process in a TC fails. 1 PH is directly attached with 1 TC.

Data Model Diagram:

[Intralogistics System](#)

[Manipulation System Ontology](#)

[Transformer Cell](#)

[Core/Production Control](#)

Issues: Not all ontologies are fully up to date and aligned

Scenario Overview

Scenario 1	Retrieval, transport, and housing disassembly
Scenario 2	Deciding on a measurement strategy
Scenario 3	Reprocessing the bevel gear
Scenario 4	Reassemble the bevel gear
Scenario 5	Plan healing and unscrewing operation

Drawio diagram:

<https://drive.google.com/file/d/1Goj2JxtH31ikqIW94YseAc3moXMb09jK/view?usp=sharing>

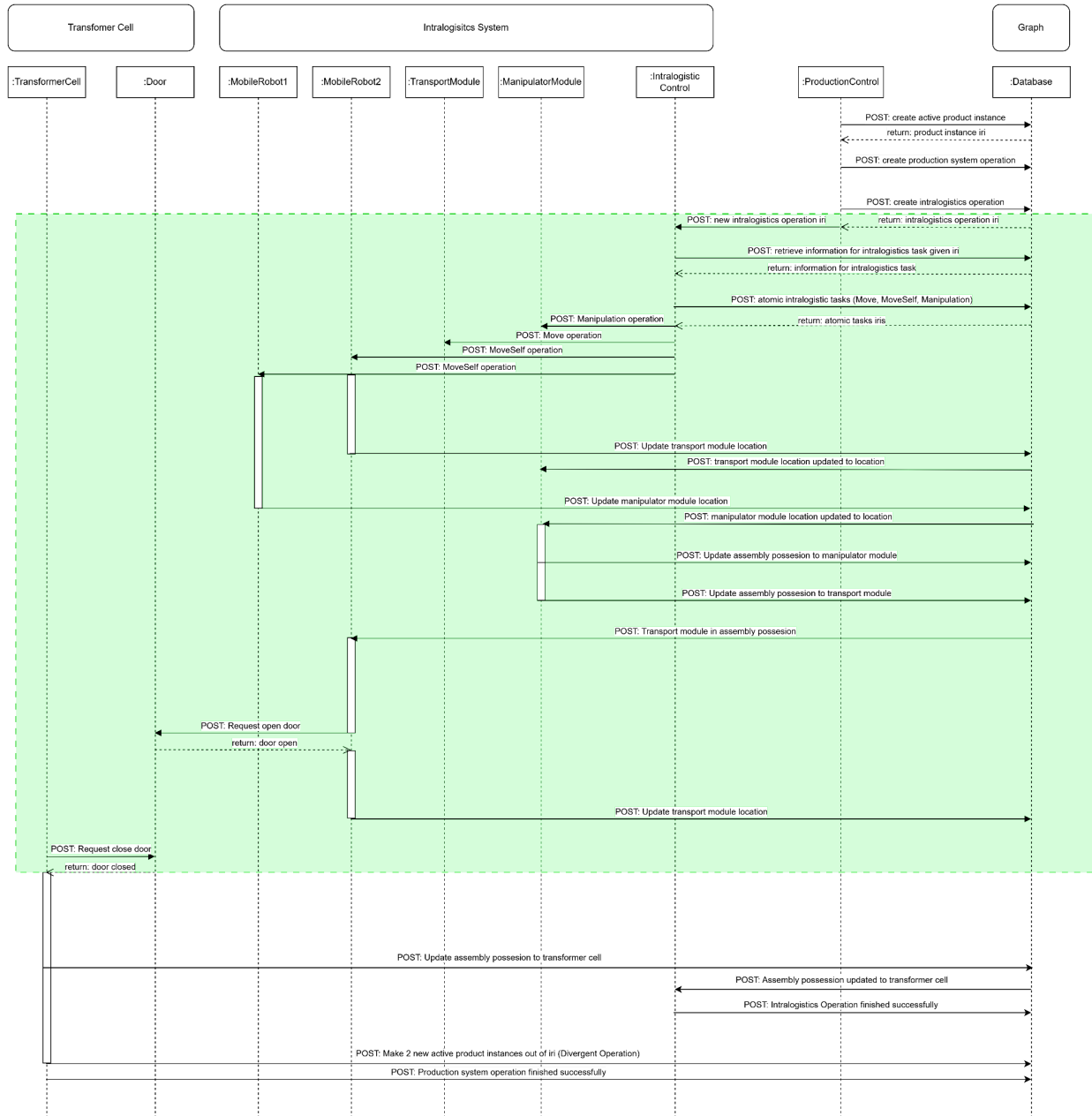
Scenario 1: (DB, PC, ILS, MS, TC)

In this scenario a newly arrived assembly that has already been initially inspected will be initialized in the knowledge graph. The assembly is to be disassembled in the transformer cell. To that end, the control logic initializes a production system operation for the disassembly in the transformer cell and an intralogistic operation for delivering the assembly from arrival point to transformer cell. The rest of the scenario is the pick up and delivery of the assembly from arrival point to transformer cell. The handover at the transformer cell and the subsequent disassembly, after which two new product instances are initialized in the knowledge graph and the old one is set to inactive. During the sequence, the product possession is updated.

Preconditions and Assumptions:

- Map with all relevant positions known
- Product placed at pickup location

Interaction Flow



Minimal reproducible bundles

[MockDatabase](#)

[MockIntralogisticSystem](#)

[MockManipulationSystem](#)

[MockProductionSystem](#)

Open issues and next steps

No currently open issues. The next steps are the implementation of the remaining steps in the sequence diagram and adapting the knowledge graphs if issues arise.

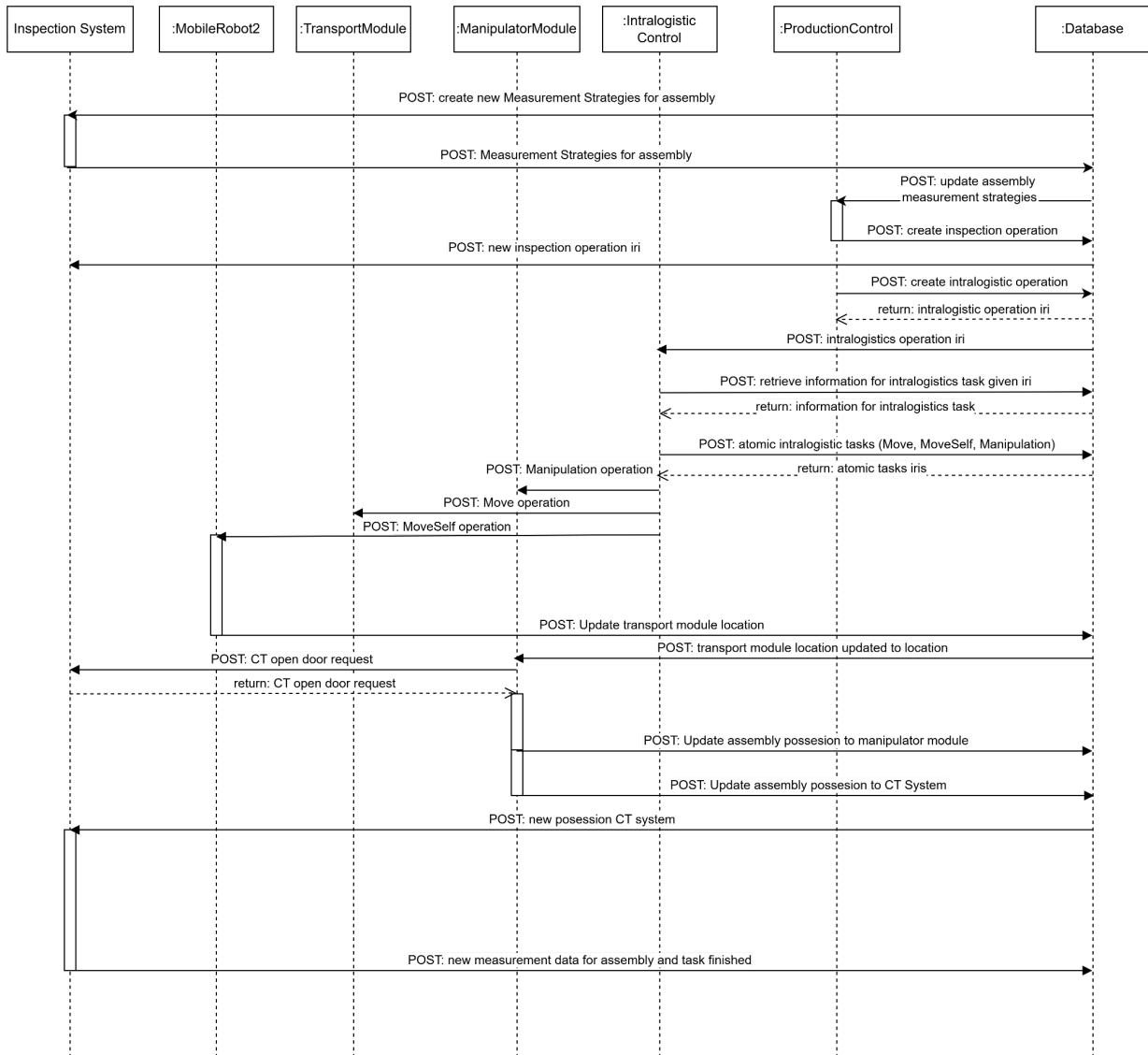
Scenario 2: (IS, ILS, PC)

In this scenario an assembly requires further measurements. For this the PC asks for potential measurement strategies from the IS. When they are returned the PC chooses a measurement strategy and creates an inspection operation for the assembly. This triggers the ILS to transport the assembly to CT machine. After arrival of the AGV, the manipulator module asks the CT to start the pick and place task and places the assembly inside of the CT. After completion the measurement is performed and the data is saved in the database.

Preconditions and Assumptions

- Map with all relevant positions known
- Assembly in storage
- Assembly with defined specification

Interaction Flow



Open issues and next steps

No currently open issues. The next steps are the implementation of the remaining steps in the sequence diagram and adapting the knowledge graphs if issues arise.

Scenario 3 (DB, PC, ILS, MS, TC):

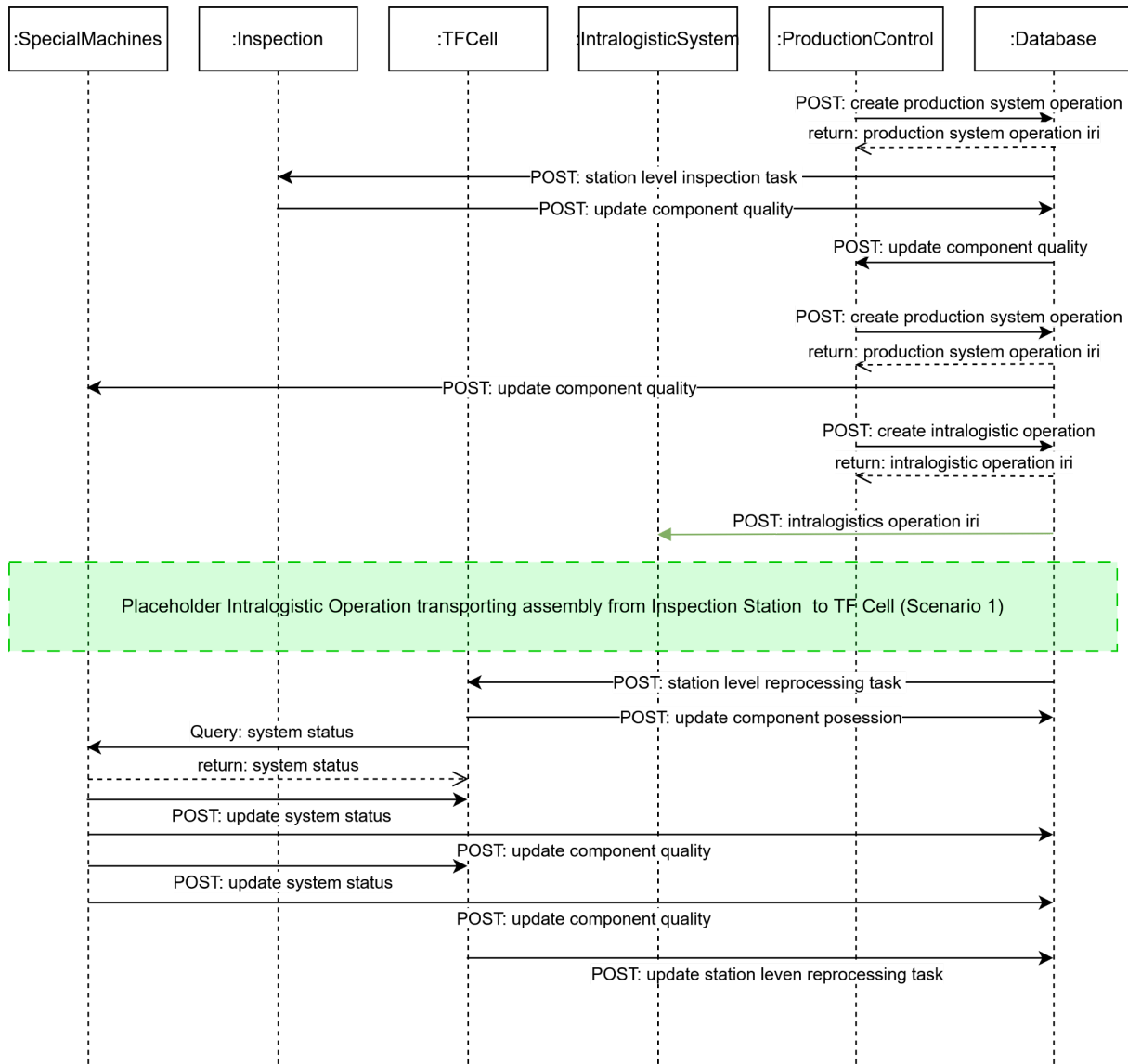
In this scenario a disassembled, initialized component starts at the inspection station for further inspection. The inspection results, saved to the knowledge graph, results in the decision to

reprocess the component in the transformer cell with the special machines. To that end, the control logic initializes a station level task (production system operation) for the reprocessing in the transformer cell and an intralogistic operation for delivering the assembly from inspection station to transformer cell. This includes the pick up and delivery of the component, the handover at the transformer cell. During the sequence, the product possession is updated. During the reprocessing the information flow is mainly between the transformer cell and the special machines: A system status is requested and returned and after each reprocessing step the system status is updated. Also, after each reprocessing step the component quality is updated in the knowledge graph. After reprocessing, the transformer cell sends an update on the station level reprocessing task (production system operation) to the knowledge graph.

Preconditions and Assumptions

- Map with all relevant positions known
- Component placed at inspection station
- Component orientation is known and not lost through the sequence

Interaction Flow



Data Exchange

Sender	Recipient	Description	Data
Database	Inspection	station level inspection task / production system level operation	product instance: {known components, ...}, decision on inspection strategy; inspection cell instance
Inspection	Database	update component quality	inspection cell instance, component/product instance, measurement results e.g. CT scan result (stl file)
Database	Production Control	update component quality	inspection cell instance, component/product instance, measurement results e.g. CT scan result (stl file)

Production Control	Database	Create production system operation	Ressource: inspection cell instance, transformer cell instance, product instance, operation: reprocess
Production Control	Database	Create transport operation	Ressource: intralogistic system instance, product instance, operation: transport, initial location, target location, (time window?)
Placeholder transport component from inspection station to tf-cell			
Database	TF-cell	station level inspection task / production system level operation	product instance; tf-cell instance, operation: reprocess
TF-cell	Database	update posession	product instance; tf-cell instance
TF-cell	Spezial machines	status query spezial machines	special machine instances, operation: reprocess, capacity query
Spezial machines	TF-cell	status update spezial machines	special machine instances, operation: reprocess, capacity answer
Database	Spezial machines	station level inspection task / production system level operation	special machine instances, operation: reprocess, product instance, measuement results (eg. CT stl), desired state (e.g. design stl), hardness etc. -> requirements
Spezial machines	TF-cell	update of component quality	special machine instance, operation: subtractive, product instance, status: done
Spezial machines	TF-cell	update of component quality	special machine instance, operation: additve, product instance, status: done
TF-cell	Spezial machines	update of component quality	inspection cell instance, component/product instance, reprocessing result e.g. success
Spezial machines	Database	update of component quality	special machine instances, component instance, operation: reprocess, sensor data, used process parameters

Minimal reproducible bundles

No new reproducible bundles compared to Szenario 1

Open issues and next steps

No currently open issues. The next steps are the critically questioning of the sequence diagram and adapting if issues arise.

Scenario 4 (DB, PC, ILS, MS, TC):

Reassembly of lower and upper part of angle grinder.

This Scenario is on hold for now. First Scenario 1,2 3 and 5 are being implemented, since the use case is quite similar to Scenario 3.

Key actors

Data Base (Knowledge graph), Production Control, Intralogistic System, Manipulation System, Transformer Cell

Interaction Flow

Minimal reproducible bundles

Open issues and next steps

Scenario 5: Plan healing of a screw removal process

This scenario is a fairly simple demonstration of **plan healing** — that is, when an operation goes wrong, the plan needs to be adjusted. During this hackathon, we focused primarily on the **communication between the Transformer Cell and the Plan Healer**, and in particular, we wanted both components to be able to **react to the information exchanged**.

To achieve this, we first tested our scenario locally, taking inspiration from the *3_modules_example.ipynb*, but modifying it so that whenever a message was sent, the relevant parties would be notified. Once this local example was working successfully, our next goal was to make **GraphDB modifications** that would **directly notify the different actors** involved using Kafka.

Key actors

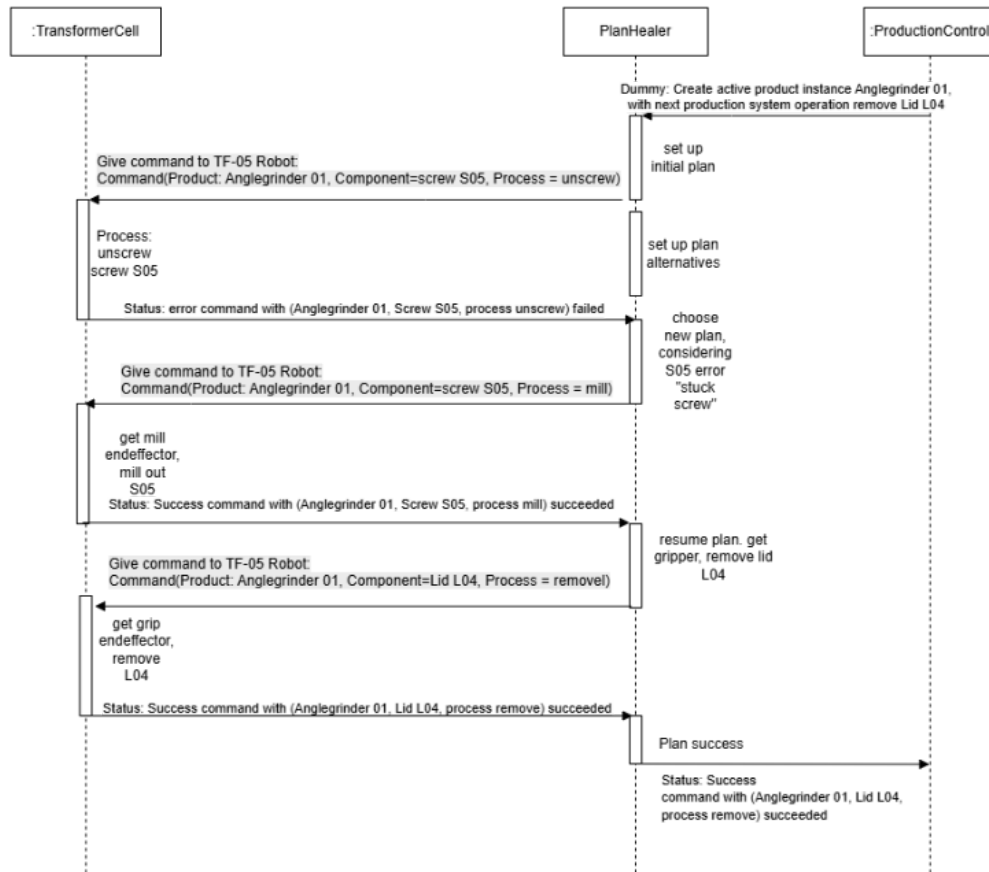
Plan Healer, Transformer Cell

Preconditions and Assumptions:

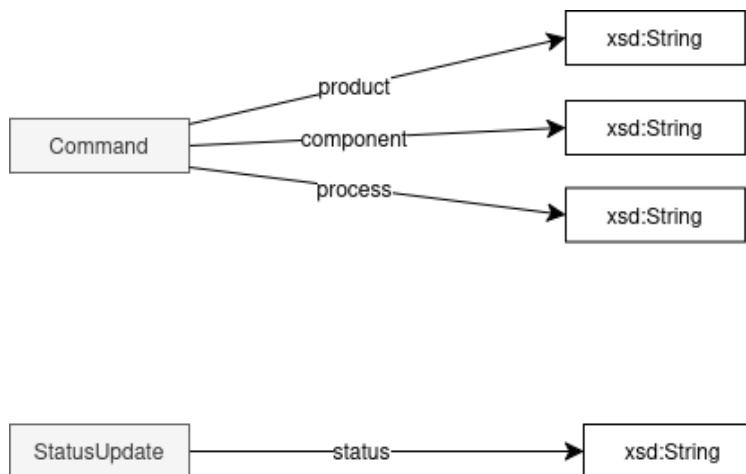
- The PC first command: remove Lid to the PH is not explicitly set in the code.
- The code is only concerned with the TC and the PH
- The Ontologies for Command and StatusUpdate are only for this scenario.

Interaction Flow

First, we defined the scenario and designed the different interactions we wanted to implement. The interactions are as follows: Production Control initiates the operation “*remove lid L04*” in Transformer Cell 5, which requires the execution of several commands — first “*unscrew screw S05*”, then “*remove lid L04*”. The first command (“*unscrew screw S05*”) is sent and executed, but unfortunately it fails. Therefore, it becomes necessary to “*mill screw S05*” in order to resume the sequence of initially planned commands. The following commands then execute successfully, and at the end, Production Control must be notified that the operation was completed successfully.



From this diagram, we observed that the Robot Transformer was expecting a **Command**, i.e. a class composed of a **product**, a **component**, and a **process**. On its side, the **Plan Heater** was expecting a **StatusUpdate** that includes the **status** attribute. We could then build the ontology associated with our scenario. We made the simple choice to define each attribute (product, component, process, and status) as a string. You can find a schematic version of our ontology.



Once we had a clear understanding of what we wanted to achieve, we took inspiration from *3_modules_example.ipynb*. The problem, however, is that the code operates using a “pull” mechanism — meaning it needs to actively request information. In our case, we don’t know when the information will be sent, so the code needs to work in “push” mode — meaning a message should be automatically sent to the relevant actors (e.g. a new command for the Transformer Cell, or a status of the command for the plan healer). This is what was implemented in the notebook [scenario_5_dummy.ipynb](#).

Minimal reproducible bundles

Either use the functional minimal example using fastAPI (Rest), see code/minimal or try to make the more complete example using kafka work. This one tries to use kafka to listen to new Commands and status updates on the knowledge graph. However the topic does not work yet. See code / wip_kafka. Set up a Semantic middleware Environment and add the files in the wip_kafka into the example_files folder.

Link to gitlab repo:

https://gitlab.kit.edu/kit/ifl/opensource/circular_factory/inf_hackaton_2025/scenario1/sc05_planhealing_unscrew.git

Open issues and next steps

The kafka example is not completed, due to issues with the kafka topic. For some reason the plan_healer_topic does not trigger. Next step for INF will be to debug this.

Next step for us might be to complete the sequence diagram flow in the kafka example as well.